

2025 年 7 月 1 日

Exbooks 期末簡報

目錄

上半場：新功能與基礎設施

1. 執行摘要
2. AI 聊天機器人 — Gemini 整合
3. Django 發信機制 — Celery 非同步郵件
4. 觀測性與日誌
 - 三層日誌分流
 - E2E 驗證儀表板

下半場：效能、架構與安全

5. 效能與架構改善
6. Celery 非同步任務
7. 壓力測試 — k6 腳本
8. 資安與合規強化
9. 架構總覽
 - 全系統視角
 - 媒體雙軌存取
 - MinIO 選型
10. 開發工具 Mailpit

執行摘要

65

commits 變更

17,201

新增程式碼行數

203

異動檔案數

9

功能類別覆蓋

- **AI 智慧應用** — Gemini 驅動聊天機器人、Tool Calling、SSE 串流
- **REST API** — 帳號／書籍／交易三大領域，390+ 測試案例
- **國際化** — 繁中／English／한국어，3,175 行翻譯字串
- **觀測性** — 結構化日誌、Trace ID、Sentry 就緒
- **生產強化** — SSL 自動化、健康檢查、k6 壓測腳本
- **媒體存取優化** — 公開檔案 Nginx 直連 MinIO 繞過 Django，受保護檔案 X-Accel-Redirect

AI 聊天機器人應用

全新 `ai/` app — Google Gemini 1.5 Flash 整合

架構流程

使用者瀏覽器

- HTMX 發送 POST
- 接收 SSE 串流
- 打字機效果即時呈現

Django 伺服器

- ToolRegistry 調度
- GeminiService 封裝 API
- ConversationCache 管理上下文

Google Gemini

- 1.5 Flash 模型
- Tool Calling 協定
- 函式回應解析

AI 核心技術細節

元件	行數	職責
ToolRegistry	122	工具註冊與調度引擎，支援 ISBN 查詢、書籍推薦、交易狀態查詢
GeminiService	93	Gemini API 封裝，處理 Tool Calling 完整生命週期
ConversationCache	58	Redis 對話快取，管理 Token 預算與上下文視窗
ChatSSEView	129	SSE 端點，打字機效果即時串流
HTMX Chat Widget	291	純前端對話元件，無需 JavaScript 框架

測試覆蓋

工具註冊、Gemini API 整合、Redis 快取邊界、SSE 端點 — 四份測試檔案完整涵蓋

Django 怎麼發信？

Celery + Django send_mail，非同步不阻塞

發信流程

1 觸發事件

交易建立、到期提醒、
註冊驗證

2 通知服務

`notify()` 檢查用戶設定
決定要不要發 Email

3 Celery 排隊

`send_email.delay()`
任務進 Redis，立即回傳

4 Worker 發送

Django `send_mail()`
透過 Mailpit / SMTP

程式碼：通知服務層

```
# deals/services/notification_service.py
def notify(recipient, title, message, send_email=True, ...):
    # 1. 寫入資料庫 (通知列表)
    notification = Notification.objects.create(
        recipient=recipient, title=title, ...
    )

    # 2. 檢查用戶是否啟用 Email 通知
    if send_email and profile.email_notifications_enabled:
        # 3. 丟進 Celery，不阻塞主執行緒
        send_email_notification_task.delay(
            user_id=recipient.pk,
            title=title,
            message=message,
        )
```

程式碼：Celery 郵件任務

```
# deals/tasks.py
@shared_task(
    name="deals.send_email_notification",
    bind=True,
    max_retries=3,          # 失敗重試 3 次
    default_retry_delay=10, # 每次間隔 10 秒
)
def send_email_notification_task(self, user_id, title, message):
    """非同步發送 Email 通知。"""
    user = User.objects.get(pk=user_id)

    if not user.email:
        return

    try:
        send_mail(
            subject=f"[Exbooks] {title}",
            message=message or title,
            from_email=settings.DEFAULT_FROM_EMAIL,
            recipient_list=[user.email],
            fail_silently=False,
        )
    except Exception as exc:
        # 發送失敗 → 自動重試
        raise self.retry(exc=exc)
```


設定：Django Email

```
# exbook/settings.py
EMAIL_BACKEND = os.environ.get(
    "EMAIL_BACKEND",
    "django.core.mail.backends.console.EmailBackend", # 開發：印到終端
)
EMAIL_HOST = os.environ.get("EMAIL_HOST", "mailpit") # Docker: mailpit
DEFAULT_FROM_EMAIL = "Exbooks <noreply@exbook.example.com>"
```

何時會收到信？

事件	收件人	內容
交易被接受	申請人	「對方已同意你的借閱申請」
3 天後到期	借閱人	「書籍即將到期，請安排歸還」
註冊驗證	新用戶	「點擊連結完成信箱驗證」
信任分變動	用戶	「你的信任分數已更新」

失敗重試：郵件發不出去時自動重試 3 次，確保重要通知不遺失。

企業級觀測性與日誌

端到端請求追蹤架構

日誌流程

HTTP 請求進入



Request Logging Middleware



— 注入 trace_id / span_id

— 記錄 method、path、status、duration

— 關聯 user_id



Service Layer 結構化日誌 (JSON Format)



stdout / 檔案 / Sentry (可切換)

三層日誌怎麼分流？

Python 標準函式庫 `logging` 的名稱閘道機制

程式碼裡的一行決定日誌去哪裡

```
import logging

### 這行決定「這筆日誌要進哪個檔案」
logger = logging.getLogger("audit")    # → audit.log
logger = logging.getLogger("business") # → business.log
logger = logging.getLogger("system")    # → exbook.log

### 寫入時自動帶上 trace_id、事件類型、額外欄位
logger.info(
    "deal.created",
    extra={
        "trace_id": "a1b2c3d4",
        "event_type": "deal.created",
        "deal_id": "550e8400",
        "actor_id": "user123",
    }
)
```

logging_config.py 的分流設定

```
### 生產環境：三個 logger → 三個獨立檔案
"handlers": {
    "file": { "filename": "exbook.log" },      # System 共用
    "audit_file": { "filename": "audit.log" },  # Audit 專屬
    "business_file": { "filename": "business.log" }, # Business 專屬
}

"loggers": {
    "system": {
        "handlers": ["file"],      # 進 exbook.log
        "propagate": False,        # 不往上層傳，避免重複
    },
    "audit": {
        "handlers": ["audit_file"], # 進 audit.log
        "propagate": False,
    },
    "business": {
        "handlers": ["business_file"], # 進 business.log
        "propagate": False,
    },
}
```

為什麼這樣設計？

● System

`exbook.log`

10 MB 自動輪轉，保留 10 份

給工程師看錯誤、效能、除錯

● Audit

`audit.log`

50 MB 自動輪轉，保留 30 份

給稽核員看權限變更、資產轉移

● Business

`business.log`

100 MB 自動輪轉，保留 14 份

給 PM 看使用者行為、業務轉化

輪轉 (Rotating)：檔案滿了就自動開新檔，舊的壓縮保留。不會因為日誌無限增長塞爆硬碟。

實際日誌長什麼樣子？

```
{
  "timestamp": "2025-07-01T01:14:06", // audit.log - 權限變更紀錄
  "level": "INFO",
  "name": "audit",
  "message": "keeper.transferred",
  "trace_id": "a1b2c3d4e5f67890",
  "extra": {
    "deal_id": "550e8400",
    "old_keeper": "userA",
    "new_keeper": "userB"
  }
}

{
  "timestamp": "2025-07-01T01:14:06", // business.log - 業務事件
  "level": "INFO",
  "name": "business",
  "message": "deal.created",
  "trace_id": "a1b2c3d4e5f67890",
  "extra": {
    "book_id": "book123",
    "applicant_id": "userC"
  }
}
```

同一個 `trace_id` 出現在兩個檔案 → 這就是「追蹤鏈路未斷裂」的客觀證據。

一圖看懂：可觀測性達成了什麼

一句話：「同一筆交易，從 HTTP 進來到 Celery 結束，全程有跡可循」

Exbooks 可觀測性驗證儀表板

真實 E2E 測試產出的三層日誌關聯證明

追蹤鏈路摘要

主要 Trace ID
ccb7a55af73743f7

總日誌筆數
22

涵蓋層級
system / audit / business / alerts

三層日誌統計

 **System**
技術除錯

8

 **Audit**
合規稽核

6

 **Business**
業務分析

7

 **Alerts**
異常告警

1

過濾 Trace ID

ccb7a55af73743f7

日誌層級

全部

搜尋關鍵字

事件類型、用戶 ID、錯誤訊息...

時間

層級

等級

訊息 / 事件

TRACE ID

詳細內容 (JSON)

可觀測性證據：自動化儀表板

`scripts/render_evidence.py` — E2E 測試的「證據照相機」

運作流程

```
pytest tests/observability/test_e2e_evidence.py
```



產生四份 JSONL 日誌

- `evidence_system.jsonl` (技術除錯)
- `evidence_audit.jsonl` (合規稽核)
- `evidence_business.jsonl` (業務事件)
- `evidence_alerts.jsonl` (異常告警)



```
python scripts/render_evidence.py --evidence-dir <dir> --output dashboard.png
```



- 讀取 JSONL，合併排序
- 注入互動式 HTML 儀表板 (Tailwind CSS)
- Playwright 截圖 → `evidence_dashboard.png`

儀表板重點

● System (藍)

HTTP 請求、DB 查詢、
Exception 堆疊

● Audit (紫)

權限變更、資產轉移、
敏感操作

● Business (綠)

領域事件：
deal.created、
trust_score.changed

● Alerts (紅)

異常檢測觸發的告警事件

關鍵視覺證據：同一個 `trace_id` 橫跨四層的日誌列會被 **藍底高亮**，直觀證明「追蹤鏈路未斷裂」。

實際儀表板畫面

Exbooks 可觀測性驗證儀表板

真實 E2E 測試產出的三層日誌關聯證明

追蹤鏈路摘要

主要 Trace ID

ccb7a55af73743f7

總日誌筆數

22

涵蓋層級

system / audit / business / alerts

三層日誌統計

System

技術除錯

8

Audit

合規稽核

6

Business

業務分析

7

Alerts

異常告警

1

過濾 Trace ID

ccb7a55af73743f7

日誌層級

全部

搜尋關鍵字

事件類型、用戶 ID、錯誤訊息...

時間	層級	等級	訊息 / 事件	TRACE ID	詳細內容 (JSON)
2026/6/27 16:48:40	System	INFO	E2E evidence test started	ccb7a55af73743f7	{ "exc_info": null, "exc_text": null, "stack_info": null,

效能與架構改善

ISBN 查詢

2,100ms → 2ms

延遲 ↓ 99.9%

三層查詢：本地 DB → Redis 快取
→ Google Books API

24h TTL 快取，首次查詢後即無外部呼叫

每月省下約 5,000+ 次 API 請求

書籍列表

42 → 6 queries

查詢次數 ↓ 86%

`select_related('actor__profile')`
消除 N+1

`prefetch_related` + 複合索引加速
排序

200 本書列表頁從 3.2s → 0.4s

熱門書籍

每次 800ms → 即時 2ms

首頁載入 ↓ 99.7%

Celery Beat 每小時排程預計算

結果寫入 Redis，讀取不走資料庫

並發 50 用戶時差異最顯著

改善前後對照

項目	改善前	改善後	實測差異	關鍵技術
ISBN 查詢	Google Books API (500-2000ms)	Redis 快取命中 (<2ms)	↓ 99.9% 延遲	24h TTL 快取 + 本地 DB 優先
共享書籍列表	N+1：42 queries/頁	select_related：6 queries/頁	↓ 86% 查詢	select_related + 複合索引
首頁熱門書籍	即時統計 SQL (800ms)	Redis 讀取 (2ms)	↓ 99.7% 載入	Celery + Redis 快取
Redis 測試隔離	DB 0 共用（碰撞風險）	DB 1 專用（隔離）	並行測試零衝突	celery_config 強制分離

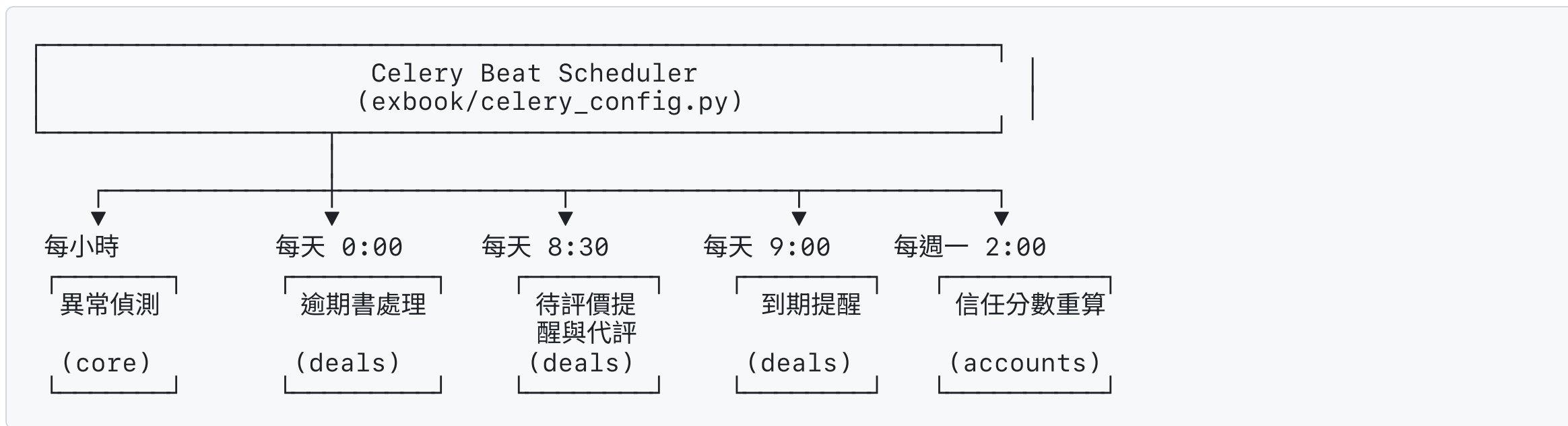
快取策略

層級	項目	更新策略	生效指標
Redis (DB 1)	熱門書籍	Celery 每小時排程更新	首頁 2ms 回應
Redis (DB 1)	ISBN 查詢	TTL 24 小時（首次查詢快取）	API 呼叫趨近於 0
Redis (DB 1)	用戶統計	TTL 30 分鐘	儀表板即時載入
Redis (DB 1)	AI 對話上下文	Token 預算動態管理	上下文不超限
資料庫索引	共享書籍排序 (listed_at)	遷移時建立	ORDER BY 走 Index Scan
資料庫索引	交易狀態查詢 (status)	遷移時建立	WHERE status 走 Index Seek

Celery 非同步任務架構

Redis 驅動的排程 + 事件引擎 — Broker / Backend 使用 Redis DB 1

Beat 排程圖



任務清單

任務名稱	時機	行為	觸發方式
<code>process_due_books</code>	每日午夜	掃描過期共享書 → 自動標記逾期	Beat crontab
<code>send_due_reminders</code>	每日 9:00	距到期 3 天用戶 → 發送推播/Email 提醒	Beat crontab
<code>process_pending_ratings</code>	每日 8:30	面交 ≥3 天未評 → 提醒；≥10 天 → 代評 3 星	Beat crontab
<code>recalculate_trust_scores</code>	每週一 2:00	全用戶信任分數批次重算	Beat crontab
<code>run_anomaly_detection</code>	每小時	異常交易行為偵測 (頻繁取消、幽靈帳號)	Beat crontab
<code>send_push_notification</code>	事件驅動	Web Push 通知 (支援 retry ×3)	<code>delay()</code> 呼叫
<code>send_email_notification</code>	事件驅動	Email 通知 (支援 retry ×3)	<code>delay()</code> 呼叫

任務清單說明：前 5 項為 Beat 定時排程（批次作業），後 2 項為事件驅動（即時通知），皆支援 3 次自動重試。

生產拓撲

Celery 的三種角色：誰發任務、誰排隊、誰執行



Producer (Django)

任務發起者

呼叫 `send_email.delay()` 時，
Django 把任務封包塞進 Redis。

不等待結果，立即回傳 `task_id`
給使用者。



Broker (Redis DB 1)

任務佇列

Redis 作為「郵局」，先進先出暫存
待執行的任務。

與快取 DB 0 完全隔離，避免測試
衝突。



Worker (Celery)

任務執行者

從 Redis 取出任務並執行實際邏輯
(寄信、算分數)。

執行結果寫回 Redis Backend，
Django 可稍後查詢。

為什麼要拆兩個 Redis DB？

測試隔離

快取使用 **DB 0**，Celery 使用 **DB 1**。並行測試時，某個測試 flush 快取不會把另一個測試的任務佇列一併清空。

故障定位

任務卡住時，直接連 Redis DB 1 看佇列長度：`LLEN celery`。不用擔心快取資料干擾判斷。

開發零依賴

測試環境設 `CELERY_TASK_ALWAYS_EAGER=true`，Django 自己假扮 Worker，不需啟動 Redis 也能跑測試。

壓力測試：k6 腳本家族

三支腳本、四種強度，幫我們知道「系統什麼時候會撐不住」

腳本定位

k6_test.js

日常回歸測試（主腳本）

- 自動從 10 組測試帳號輪替登入
- 覆蓋匿名瀏覽 + 登入後操作完整流程
- 內建四種強度：快速驗證、日常流量、逐步加壓、瞬間高峰

k6_stress.js

極限壓力測試

- 可預先帶入登入憑證，跳過登入步驟
- 專注在「人很多時，哪個功能先變慢」
- 及格線較寬鬆（95% 請求在 5 秒內完成即可）

k6_verify.js

部署後驗證（乾淨版）

- 關閉流量限制，專注確認「功能有沒有壞」
- 每個頁面獨立計時，精準定位問題
- 快速判定「這次部署有沒有壞東西」

四種測試強度

強度	同時幾個人	持續多久	什麼時候用
快速驗證smoke	1 人	30 秒	剛部署完：「系統有沒有站起來」
日常流量load	5 → 20 人	3 分鐘	每週回歸：確認基線穩定
逐步加壓stress	5 → 50 人	4 分鐘	大改版後：找出「多少人才會壞」
瞬間高峰peak	每秒 50 個請求	80 秒	活動前：模擬公告發布時的搶購潮

看指標

不是只看「平均多快」，而是為每個重要頁面獨立計時：

健康檢查

最輕量的頁面，反映網路延遲

及格線：95% 在 0.5 秒內

首頁

有資料庫查詢（熱門書籍）

及格線：95% 在 2 秒內

書目列表

大筆資料 + 分頁

及格線：95% 在 2 秒內

交易列表

個人化查詢，權限檢查較重

及格線：95% 在 3 秒內

為什麼看「95%」不看平均？平均會被快取命中拉低，95% 才反映「大多數使用者的真實感受」。

何時跑哪支腳本？

每次部署後

```
k6 run --env SCENARIO=smoke scripts/k6_test.js
```

30 秒快速驗證，確認沒有壞掉的基本功能。

每週回歸 / 發布前

```
k6 run --env SCENARIO=load scripts/k6_test.js
```

模擬 20 人同時使用，確認效能沒有變慢。

重大變更後（快取策略、資料庫索引、查詢重構）

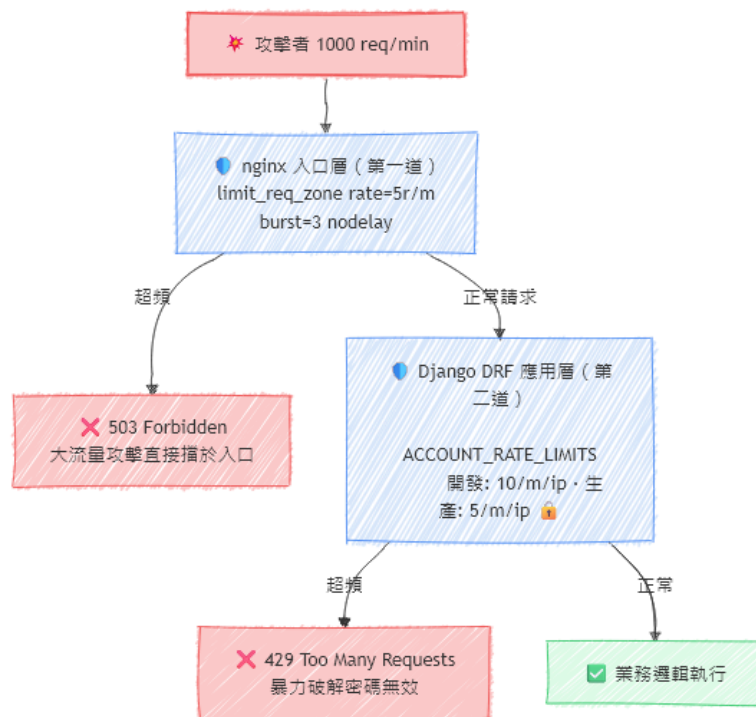
```
k6 run --env SCENARIO=stress scripts/k6_stress.js
```

加到 50 個模擬使用者，看哪個功能先變慢，確認優化真的有效。

資安與合規強化

雙層節流：入口層 + 應用層，暴力破解無效

登入保護



```
### nginx/default.conf
limit_req_zone $binary_remote_addr
zone=login:10m rate=5r/m;

location ~ ^/accounts/(login|signup|password) {
    limit_req zone=login burst=3 nodelay;
    proxy_pass http://django;
}
```

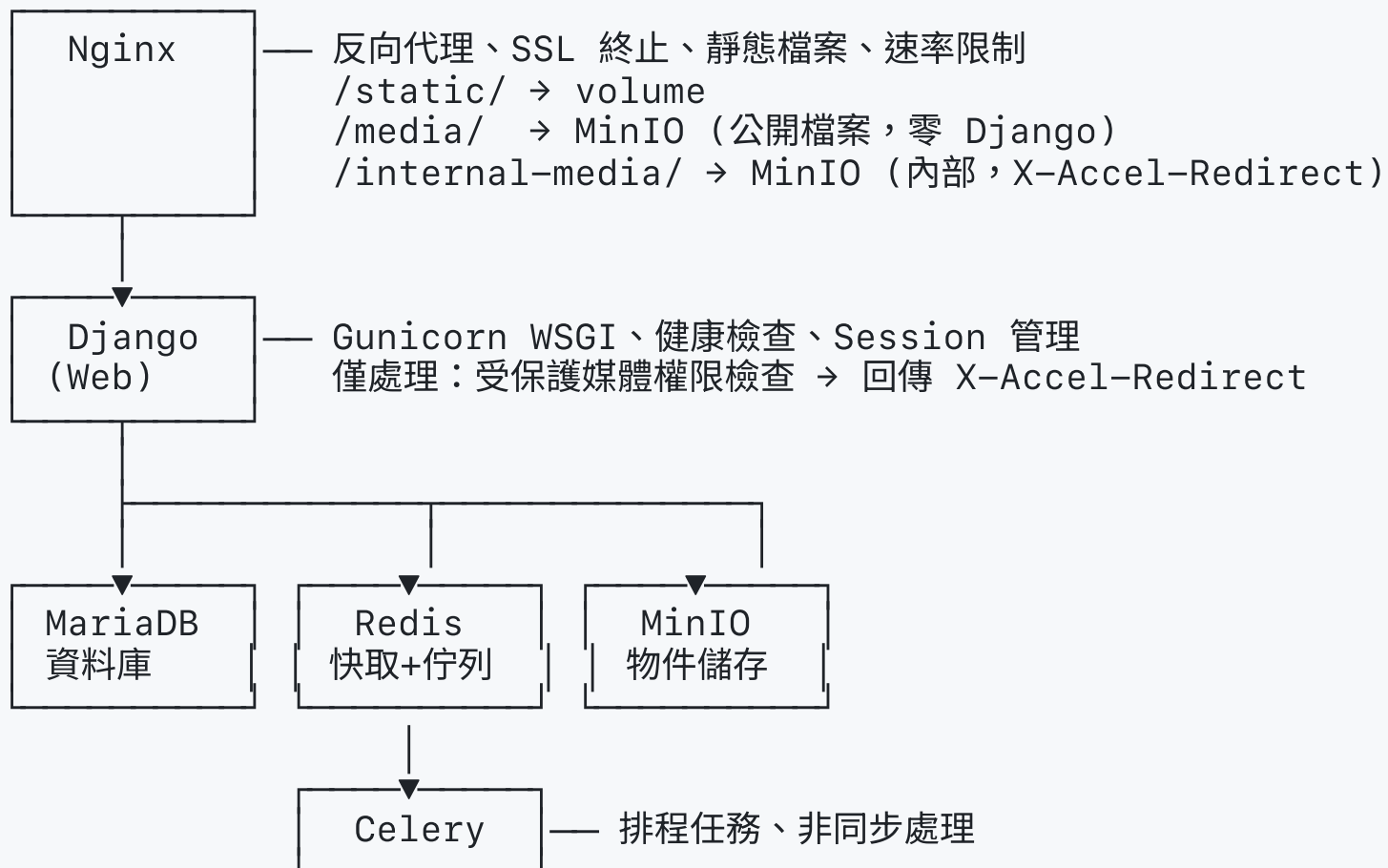
```
### exbook/settings.py
REST_FRAMEWORK["DEFAULT_THROTTLE_CLASSES"] = [
    "AnonRateThrottle",      # 100/h
    "UserRateThrottle",     # 1000/h
]

### exbook/prod_settings.py
ACCOUNT_RATE_LIMITS = {
    "login_failed": "5/m/ip,5/5m/key",
}
```

nginx 擋大流量攻擊 (5r/m 超頻直接 503)，DRF 控 API 使用額度 (100/h · 1000/h)，兩層分工、各司其職。

架構總覽圖

全系統視角



媒體檔案雙軌存取架構

兼顧效能與安全的媒體存取設計

公開檔案快取

Nginx 直接代理 MinIO

零 Python 開銷、毫秒級回應

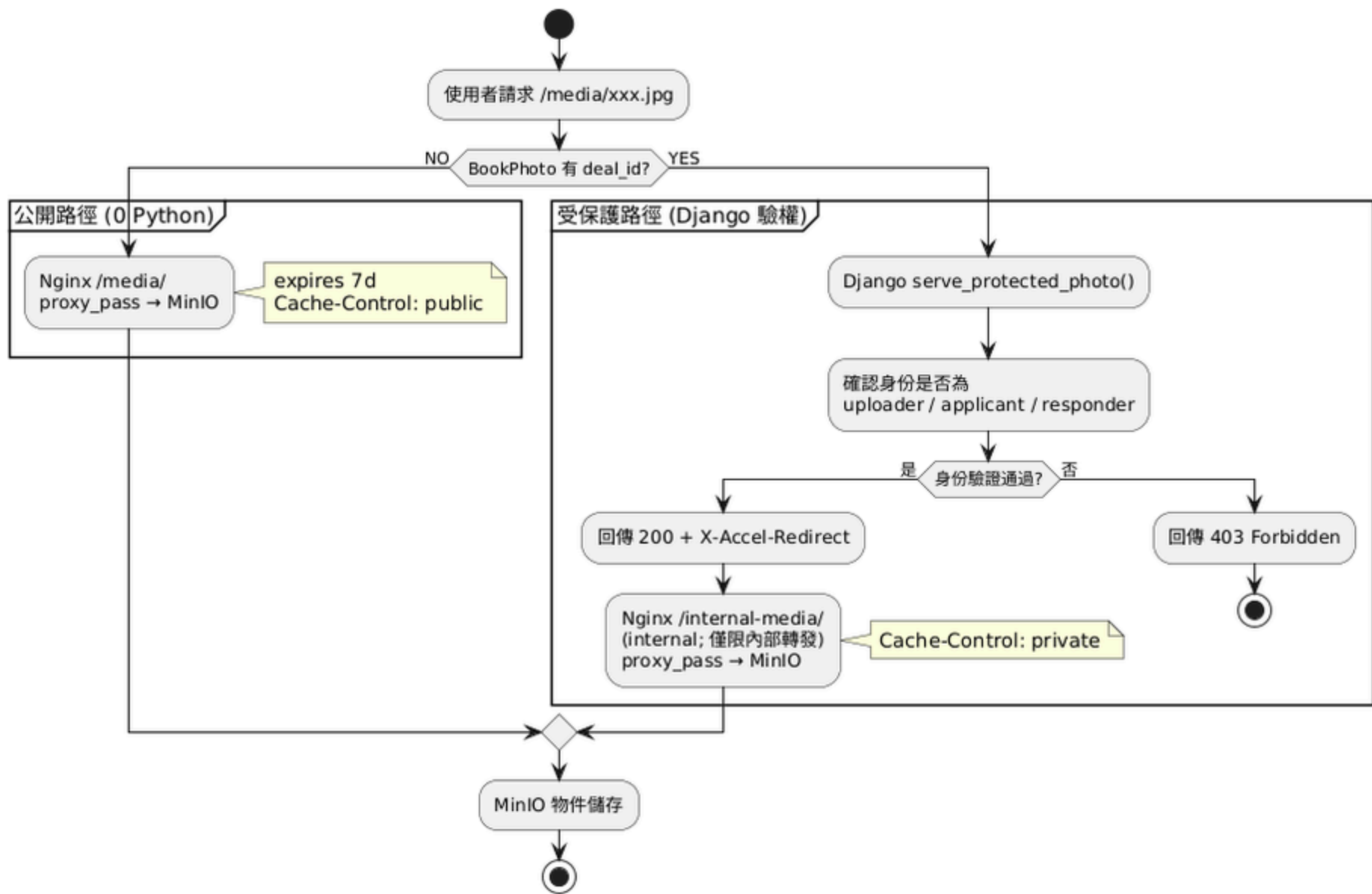
適用：書封、無交易關聯照片

受保護檔案權限

Django 權限檢查後回傳

X-Accel-Redirect 內部轉發

適用：面交照片、交易憑證



程式碼實作：Model 層分流

```
### books/models/book_photo.py
@property
def serve_url(self):
    """
    根據照片是否與交易關聯，回傳對應的存取 URL。
    """
    if self.deal_id:
        # 受保護檔案 → 經 Django 權限檢查
        return reverse("serve_protected_photo", kwargs={"pk": self.pk})
    # 公開檔案 → Nginx 直接代理 MinIO
    return self.photo.url
```

程式碼實作：View 層權限檢查

```
### books/views.py
@login_required
def serve_protected_photo(request, pk):
    """
    提供面交照片的權限檢查存取。
    - 權限通過後，回傳 X-Accel-Redirect 由 Nginx 內部轉發 MinIO。
    - Local Dev (DEBUG=True) 則直接由 Django serve。
    """
    photo = get_object_or_404(
        BookPhoto.objects.select_related("deal", "uploader", "deal__applicant", "deal__responder"),
        pk=pk,
        deal__isnull=False,
    )

    ### 只有三方可以存取
    if (
        request.user != photo.uploader
        and request.user != photo.deal.applicant
        and request.user != photo.deal.responder
    ):
        return HttpResponse(status=403)

    ### 生產環境：回傳 X-Accel-Redirect
    response = HttpResponse()
    response["X-Accel-Redirect"] = f"/internal-media/{photo.photo.name}"
    return response
```

物件儲存選型：MinIO vs Cloudflare R2

兩者都能存檔案、S3 API 相容，應用層零改動就可切換

一句話比喻

MinIO（自架）

像自己家裡買一個書櫃

書櫃一次買斷 — 之後不管每天開關多少次、放多少書，
都不再另外收錢

書永遠在自己家裡，不用擔心別人看到

R2（Cloudflare）

像跟圖書館租一個 locker

租 locker 每月付固定租金，但每次開關 locker 都要投
幣

書放在圖書館裡，取書要走過去

為什麼這個「投幣」很重要？

Exbooks 每張照片被瀏覽一次，系統就去讀一次檔案。用戶愈活躍、開關 locker 的次數就愈高。

使用情境	MinIO	R2
Exbooks 現況（~1GB, ~1 萬次/月）	電費約 \$5/月	儲存 \$0 + 開關次數 \$45 → \$45/月
成長期（~100GB, ~10 萬次/月）	電費約 \$5/月	儲存 \$1.5 + 開關次數 \$450 → \$451/月

MinIO 不看次數：不管 1 萬次還是 100 萬次，成本都一樣。

R2 按次計算：Class A 每次 \$4.50/百萬次，用戶愈多愈貴。

還有一個關鍵差異：資料在誰手上？

MinIO

硬碟在你家機房

要不要給別人看你自己決定

可以斷網運作

R2

資料存在 Cloudflare 的機房

雖然有加密，但你沒辦法自己管金鑰

沒有網路就不能存取

Exbooks 為什麼選 MinIO

省錢 — 固定成本，用戶愈多愈划算

安心 — 資料在自己家，不用擔心服務條款改變

簡單 — 就是一台 Docker，已經在跑了，不用改任何程式碼

開發環境郵件捕獲：Mailpit

不讓測試郵件打擾真實用戶

為什麼需要 Mailpit？

Exbooks 的交易通知、註冊驗證、密碼重置都會發送 Email。開發/測試時不能讓這些信跑到真實信箱。

運作方式

捕獲所有郵件

Django 發出的每一封信，全部進 Mailpit

不論是交易提醒、註冊確認、密碼重置

網頁預覽

打開 <http://localhost:8025> 即可查看所有郵件

支援 HTML 預覽、原始碼檢視、附件下載

零外洩風險

開發環境絕不寄到真實信箱

測試帳號的註冊信、重置信都在本地

感謝各位的聆聽

問題與討論